

SOFTWARE REQUIREMENTS SPECIFICATION

Real-Time Collaborative Workspace Platform

Google Docs + Figma Inspired

Field	Details
Document Version	1.0
Project Type	Final Year / Capstone Project
Level	Advanced — High Placement Value
Target Roles	SDE Intern, Product-Based Companies, 8–20+ LPA Placements
Prepared By	Development Team
Date	10 June 2026
Status	Draft — Under Review

Table of Contents

1. Introduction & Overview
2. System Overview
3. Stakeholders & User Classes
4. Functional Requirements
 - 4.1 Authentication Module
 - 4.2 Workspace Module
 - 4.3 Document Collaboration Module
 - 4.4 Canvas / Design Board Module
 - 4.5 Real-Time Collaboration Engine
 - 4.6 Communication Module
5. Non-Functional Requirements
6. System Architecture
7. Database Design
8. API Design
9. Security Requirements
10. Tech Stack Justification
11. Milestones & Timeline
12. Glossary

1. Introduction & Overview

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the Real-Time Collaborative Workspace Platform — a full-stack web application that enables teams to collaborate on documents and design boards simultaneously, in real time.

This document serves as the authoritative reference for the development team, academic evaluators, and technical interviewers assessing project quality and architectural depth.

1.2 Scope

The platform encompasses two primary workspaces:

- Document Workspace: A rich-text collaborative editor inspired by Google Docs, supporting simultaneous multi-user editing, cursor tracking, comments, version history, and auto-save.
- Design Board Workspace: An infinite-canvas drawing and design tool inspired by Figma, supporting shapes, text, sticky notes, freehand drawing, layers, and real-time multi-user collaboration.
- Workspace Management: Team creation, member invitation, role-based access control, and activity feeds.
- Communication Layer: In-app chat, @mentions, and real-time notifications.

1.3 Definitions & Acronyms

Term	Definition
OT	Operational Transform — algorithm for real-time conflict resolution in collaborative text editing
CRDT	Conflict-free Replicated Data Type — data structure ensuring eventual consistency
JWT	JSON Web Token — stateless authentication mechanism
RBAC	Role-Based Access Control — permission model based on user roles
Socket.IO	WebSocket library enabling bidirectional, real-time communication
Redis	In-memory data store used for caching, presence tracking, and pub/sub
Fabric.js	JavaScript canvas library for rich interactive canvas objects
CDN	Content Delivery Network for serving static assets
SPA	Single Page Application — frontend served as one HTML document
REST	Representational State Transfer — stateless API design pattern

2. System Overview

2.1 Product Perspective

The platform is a cloud-hosted SPA delivered through a browser. It combines the collaborative text-editing paradigm of Google Docs with the canvas-based design experience of Figma into a single unified workspace, removing the need to switch between tools.

2.2 Product Functions — Summary

Module	Key Capabilities
Authentication	Register, Login, Google OAuth, Password Reset, Profile Management
Workspace	Create/manage teams, invite members, assign roles, activity feed
Document Editor	Rich text, real-time co-editing, cursors, comments, version history, auto-save
Design Board	Infinite canvas, shapes, text, sticky notes, layers, zoom/pan, real-time sync
Real-Time Engine	Live cursors, presence indicators, OT/CRDT sync, conflict resolution, reconnection
Communication	Team chat, @mentions, in-app notifications, activity logs

2.3 User Classes

Role	Description	Permissions
Owner	Workspace creator	Full control — all CRUD, billing, deletion
Admin	Workspace administrator	Manage members, content, settings (except billing)
Editor	Active contributor	Create, read, update documents & boards
Commenter	Reviewer	View content, add comments only
Viewer	Read-only guest	View documents and boards

4. Functional Requirements

4.1 Authentication Module

FR-AUTH-01: User Registration

- Accept full name, email address, and password
- Validate email format and uniqueness
- Hash password using bcrypt with minimum cost factor of 12
- Send email verification link (expires in 24 hours)
- Return JWT access token (15-min TTL) and refresh token (7-day TTL) on success

FR-AUTH-02: User Login

- Authenticate with email + password or Google OAuth 2.0
- Implement rate limiting: max 5 failed attempts per 15 minutes per IP
- Issue new access and refresh token pair on success
- Log login event with timestamp, IP, and device info

FR-AUTH-03: Google OAuth 2.0

- Integrate with Google OAuth via passport.js
- Auto-register new users on first Google login
- Link Google account to existing account if email matches

FR-AUTH-04: Password Reset

- Send password reset link to verified email
- Reset token must expire within 1 hour
- Invalidate all existing sessions on password change

FR-AUTH-05: Profile Management

- Allow update of display name, avatar image, bio, and timezone
- Store avatar in Cloudinary/S3, save URL reference in MongoDB
- Support account deletion with cascade cleanup

4.2 Workspace Module

FR-WS-01: Create Workspace

- Unique workspace slug (URL-safe, alphanumeric, max 50 chars)
- Workspace name, description, and logo upload
- Creator automatically assigned Owner role

FR-WS-02: Member Invitation

- Invite by email with configurable role
- Generate invitation link with 7-day expiry
- Email notification sent to invitee
- Invitation accepted via link or in-app notification

FR-WS-03: Role Management

- Owner can promote/demote any member
- Admin can manage Editor/Commenter/Viewer roles
- Role changes propagate in real-time via Socket.IO

FR-WS-04: Workspace Activity Feed

- Log all significant events: document creation, member joins, role changes, etc.
- Display in chronological order with actor avatar, action, and timestamp
- Filterable by event type

4.3 Document Collaboration Module

FR-DOC-01: Create & Manage Documents

- Create document within workspace with title and optional template
- Organize documents into folders
- Soft-delete with 30-day recovery window

FR-DOC-02: Rich Text Editor

- Formatting: bold, italic, underline, strikethrough, inline code
- Headings (H1–H6), blockquote, horizontal rule
- Ordered and unordered lists with nesting
- Tables with resizable columns
- Image embedding (upload to S3)
- Code blocks with syntax highlighting
- Hyperlinks with preview on hover

FR-DOC-03: Real-Time Co-Editing

- Apply Operational Transform (OT) or Y.js CRDT for conflict-free concurrent edits
- Broadcast delta operations to all connected clients via Socket.IO
- Converge to identical document state across all clients
- Handle network partition and reconnection gracefully

FR-DOC-04: Live Cursor Tracking

- Broadcast cursor position and selection range of each collaborator
- Render collaborator cursors in distinct color with name label
- Hide cursor of disconnected collaborator within 3 seconds

FR-DOC-05: Comments

- Anchor comments to specific text ranges
- Threaded replies per comment
- Resolve/reopen comment threads
- @mention in comment triggers notification to mentioned user

FR-DOC-06: Version History

- Snapshot document state every 5 minutes and on explicit save
- List version history with author, timestamp, and change summary
- Preview any historical version in read-only mode
- Restore document to any historical version

FR-DOC-07: Auto-Save & Sharing

- Auto-save triggered every 3 seconds after last keystroke
- Visual indicator showing save status (saving / saved / error)
- Generate shareable link with configurable permission level
- Password-protect shared link (optional)

4.4 Canvas / Design Board Module

FR-CVS-01: Infinite Canvas

- Smooth pan via click-and-drag on empty canvas
- Zoom 10%–400% via scroll wheel or pinch gesture
- Mini-map navigator in corner for spatial context
- Snapping grid with configurable spacing

FR-CVS-02: Shape Tools

- Rectangle, circle/ellipse, triangle, polygon, line, arrow
- Freehand pen/pencil tool
- Properties panel: fill, stroke color, stroke width, opacity, corner radius
- Resize handles with aspect-ratio lock (Shift key)
- Rotation handle with degree snap

FR-CVS-03: Text & Sticky Notes

- Double-click to add text anywhere on canvas

- Font family, size, weight, alignment, color
- Sticky note with background color options

FR-CVS-04: Object Management

- Select single or multiple objects (Shift-click or drag-to-select)
- Cut, copy, paste, duplicate, delete
- Align and distribute selected objects
- Group and ungroup objects

FR-CVS-05: Layer Management

- Layers panel listing all objects in z-order
- Reorder via drag in layers panel
- Lock layer to prevent accidental editing
- Show/hide individual layers

FR-CVS-06: Real-Time Canvas Collaboration

- Broadcast object position, resize, and property changes via Socket.IO
- Show collaborator cursors as named pointers on canvas
- Optimistic updates with server reconciliation
- Undo/redo per user (Ctrl+Z / Ctrl+Y) without affecting collaborators

4.5 Real-Time Collaboration Engine

FR-RT-01: Presence System

- Track connected users per document/board via Redis
- Broadcast join/leave events to all collaborators
- Show avatar stack of active collaborators in toolbar
- User status: Active, Idle (3-min inactivity), Offline

FR-RT-02: Conflict Resolution

- Document text: Y.js CRDT (preferred) or OT via operational transforms
- Canvas objects: Last-Write-Wins with object-level locking on drag
- Merge queue for offline edits submitted on reconnection

FR-RT-03: Real-Time Notifications

- Emit notification events for: @mention, comment, role change, share
- Persist unread notifications in MongoDB
- In-app notification bell with unread count badge
- Mark as read individually or bulk-clear

4.6 Communication Module

FR-COMM-01: Team Chat

- Per-workspace chat channels
- Send text messages with markdown support
- Attach files (images, PDFs) via S3 upload
- Emoji reactions on messages

FR-COMM-02: Mentions & Notifications

- @username autocomplete in chat and document comments
- Mentioned user receives in-app notification and email

FR-COMM-03: Activity Logs

- Timestamped audit trail of all workspace events
- Exportable as CSV for Owner/Admin roles

5. Non-Functional Requirements

5.1 Performance

Metric	Target
Real-time sync latency (p95)	< 200 ms
API response time (p95)	< 500 ms
Canvas render frame rate	> 60 fps on modern hardware
Auto-save interval	Every 3 seconds after last edit
Page load time (LCP)	< 2.5 seconds on 4G

5.2 Scalability

Dimension	Target
Concurrent WebSocket connections	10,000+
Documents per workspace	Unlimited (paginated lists)
Canvas objects per board	10,000+ with spatial indexing
Horizontal scaling	Stateless Node.js instances + Redis pub/sub for Socket.IO

5.3 Security

- All data in transit encrypted via TLS 1.3 (HTTPS + WSS)
- Passwords hashed with bcrypt (cost 12+), never stored in plain text
- JWT tokens signed with RS256 (asymmetric), rotated on each refresh
- RBAC enforced at both API middleware and Socket.IO event handler level
- Input sanitization to prevent XSS; parameterized queries to prevent injection
- CORS configured to whitelist only known frontend origins
- Helmet.js for HTTP security headers
- Rate limiting on all auth endpoints

5.4 Reliability

Requirement	Target
System availability	99.9% uptime (< 8.7 hrs downtime/year)
Automatic WebSocket reconnection	Exponential backoff, max 5 retries
Data durability	MongoDB replica set with 3 nodes
Backup frequency	Daily automated backups with 30-day retention

5.5 Usability

- Responsive design supporting viewport widths from 768px to 2560px
- Cross-browser support: Chrome 110+, Firefox 110+, Safari 16+, Edge 110+
- Keyboard shortcuts for all common editing actions
- Accessible color contrast ratios (WCAG AA minimum)
- Optimistic UI updates — no visible loading for most user actions

6. System Architecture

6.1 High-Level Architecture Layers

Layer	Technology	Responsibility
Client (SPA)	React + TypeScript + Tailwind CSS	Render UI, manage local state, emit/receive Socket.IO events
API Gateway	Express.js	Route REST requests, authenticate JWT, apply rate limits
Real-Time Server	Socket.IO (on same Node.js process)	Manage WebSocket rooms, broadcast deltas, handle presence
Business Logic	Node.js Services	Workspace, Document, Canvas, Auth service layers
Primary DB	MongoDB (Replica Set)	Persist documents, users, workspaces, canvases
Cache & Presence	Redis	Active user presence, Socket.IO adapter, session cache
File Storage	AWS S3 / Cloudinary	Store uploaded images, avatars, attachments

6.2 Real-Time Data Flow

The sequence of a collaborative edit operation:

1. User makes an edit (keystroke/canvas move) in the React client.
2. Client applies the operation locally (optimistic update) and emits a Socket.IO event to the server room.
3. Server validates the operation against the current document revision.
4. OT/CRDT transform is applied if concurrent operations are detected.
5. Transformed operation is broadcast to all other clients in the room.
6. Each receiving client applies the operation to its local state.
7. Persisted to MongoDB asynchronously every auto-save interval.

6.3 Socket.IO Room Strategy

- Each document/board has a dedicated Socket.IO room keyed by its ID.
- User joins room on opening a document; leaves on tab close or navigation.
- Redis adapter enables multi-instance Socket.IO broadcasting.
- Presence data stored in Redis with TTL matching heartbeat interval (30s).

7. Database Design

7.1 Collections Overview

Collection	Purpose	Key Fields
users	User accounts	_id, email, passwordHash, name, avatar, googleId, createdAt
workspaces	Team workspaces	_id, slug, name, ownerId, members[{userId, role}], createdAt
documents	Document metadata + content	_id, workspaceId, title, content (Quill delta), authorId, version, lastSavedAt
doc_versions	Version snapshots	_id, documentId, content, authorId, createdAt
boards	Canvas metadata + object list	_id, workspaceId, title, objects[], authorId, version
comments	Document comments	_id, documentId, anchorRange, authorId, text, resolved, replies[]
messages	Chat messages	_id, workspaceId, channelId, authorId, text, attachments[], createdAt
notifications	User notifications	_id, userId, type, payload, read, createdAt
invitations	Pending invites	_id, workspaceId, email, role, token, expiresAt
activity_logs	Audit trail	_id, workspaceId, actorId, action, target, metadata, createdAt

8. API Design (REST)

8.1 Authentication Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login with email/password
POST	/api/auth/google	Google OAuth callback
POST	/api/auth/refresh	Refresh access token
POST	/api/auth/logout	Invalidate refresh token
POST	/api/auth/forgot-password	Send reset email
POST	/api/auth/reset-password	Reset with token

8.2 Workspace Endpoints

Method	Endpoint	Description
POST	/api/workspaces	Create workspace
GET	/api/workspaces/:id	Get workspace details
PATCH	/api/workspaces/:id	Update workspace
DELETE	/api/workspaces/:id	Delete workspace
POST	/api/workspaces/:id/invite	Invite member
PATCH	/api/workspaces/:id/members/:userId	Update member role
DELETE	/api/workspaces/:id/members/:userId	Remove member
GET	/api/workspaces/:id/activity	Get activity feed

8.3 Document Endpoints

Method	Endpoint	Description
POST	/api/documents	Create document
GET	/api/documents/:id	Get document + current content
PATCH	/api/documents/:id	Update metadata
DELETE	/api/documents/:id	Soft-delete
GET	/api/documents/:id/versions	List version history
POST	/api/documents/:id/restore/:versionId	Restore version
POST	/api/documents/:id/share	Generate share link

Method	Endpoint	Description
POST	/api/documents/:id/comments	Add comment
PATCH	/api/documents/:id/comments/:cid	Resolve/edit comment

8.4 Socket.IO Events

Event (Client → Server)	Payload	Description
doc:join	{ documentId }	Join document room
doc:operation	{ documentId, op, revision }	Broadcast OT operation
doc:cursor	{ documentId, range, color }	Broadcast cursor position
canvas:object-update	{ boardId, object }	Broadcast object change
canvas:cursor	{ boardId, x, y }	Broadcast canvas pointer
presence:heartbeat	{ workspaceId }	Keep presence alive
chat:message	{ channelId, text }	Send chat message

9. Security Requirements

Threat	Mitigation
Unauthorized API access	JWT middleware on all protected routes; RS256 signature verification
Privilege escalation	RBAC checks in service layer, not just route middleware
XSS via rich text	Sanitize HTML with DOMPurify before rendering; CSP headers via Helmet.js
Brute-force login	express-rate-limit: 5 requests / 15 min on auth routes; account lockout
Token theft	Short-lived access tokens (15 min); refresh tokens stored in httpOnly cookies
Data exposure in transit	TLS 1.3 enforced; HSTS header; WSS (not WS) for WebSockets
Insecure file uploads	Validate MIME type and size server-side; store in S3 (not public server disk)
MongoDB injection	Mongoose schema validation; never build queries with raw user input

10. Tech Stack Justification

Technology	Role	Why Chosen
React 18 + TypeScript	Frontend SPA	Component model ideal for complex, stateful collaborative UIs; TypeScript prevents runtime errors
Tailwind CSS	Styling	Utility-first; highly consistent design system without writing custom CSS
Fabric.js / Konva.js	Canvas rendering	Handles complex object model, hit-testing, and event delegation on HTML5 Canvas
Y.js	CRDT library	Battle-tested CRDT implementation; integrates with Quill and Fabric.js
Quill.js / TipTap	Rich text editor	Extensible editor with Delta operations compatible with OT/CRDT
Node.js + Express	Backend API	Non-blocking I/O ideal for high-concurrency real-time workloads
Socket.IO	Real-time layer	Abstraction over WebSocket with fallback, rooms, namespaces, and adapter support
MongoDB	Primary database	Flexible document model suits variable-structure document deltas and canvas objects
Redis	Cache + Pub/Sub	Sub-millisecond reads for presence data; Socket.IO Redis adapter enables horizontal scaling
JWT (RS256)	Auth tokens	Stateless; RS256 allows public-key verification without sharing secret
Cloudinary / AWS S3	File storage	Scalable, CDN-backed object storage; avoids storing blobs in DB
Docker + Nginx	Deployment	Containerized services; Nginx as reverse proxy with SSL termination

11. Milestones & Development Timeline

Phase	Duration	Deliverables
Phase 1 — Foundation	Week 1–2	Project setup, DB schema, Auth module (register, login, Google OAuth), basic workspace CRUD
Phase 2 — Document Core	Week 3–4	Quill editor integration, document CRUD, basic Socket.IO sync (no OT yet), auto-save
Phase 3 — Real-Time Collaboration	Week 5–6	Y.js CRDT integration, live cursors, presence system via Redis, conflict resolution
Phase 4 — Canvas Board	Week 7–8	Fabric.js canvas, shape/text tools, layer management, canvas real-time sync
Phase 5 — Workspace Features	Week 9–10	Role-based access, invitations, comments, version history, notifications
Phase 6 — Communication	Week 11	Team chat, @mentions, activity feed, activity log export
Phase 7 — Polish & NFRs	Week 12	Performance optimization, security hardening, responsive UI, cross-browser testing
Phase 8 — Deployment & Docs	Week 13–14	Docker setup, CI/CD pipeline, deployment, API documentation, project report

12. Glossary

Term	Definition
Delta Operation	An atomic description of a change (insert/delete/retain) applied to a document
Operational Transform	Algorithm ensuring two concurrent operations can be transformed to achieve consistent document state
CRDT	Data structure designed so that concurrent updates from multiple clients always converge to the same state without central coordination
Presence	Real-time awareness of which users are currently active in a document or board
Heartbeat	Periodic signal sent by the client to indicate it is still connected and active
Soft Delete	Marking a record as deleted without removing it from the database, allowing recovery
Optimistic Update	Applying a UI change immediately before server confirmation, then reverting if the server rejects it
Room (Socket.IO)	A named channel that a subset of connected sockets can join to receive targeted broadcasts
Revision	A monotonically increasing integer representing the current version of a document or board
OT Transform	Function that takes two concurrent operations and produces versions of each that, when applied, produce the same final document state

End of Document — SRS v1.0 | Real-Time Collaborative Workspace Platform